Author: Micheal Lanham

SECTION 1

MCP Architecture & Core Components

• **The Three-Part Architecture:** The MCP ecosystem separates concerns into three distinct entities:
• **MCP Client:** The AI agent, desktop app (e.g., Claude Desktop, VS Code), or LLM framework that connects to servers and discovers available capabilities.
• **MCP Server:** The protocol handler that exposes tools, processes JSON-RPC requests, and abstracts backend complexity.
• **Underlying Service:** The target application, database, API, or external file system being accessed.
• **The Three Capability Primitives:** MCP servers expose functionality using three primitives:
• **Tools:** Actionable, model-controlled functions that agents execute autonomously to perform work.
• **Resources:** Application- or user-controlled data sources (such as files, databases, or API feeds) for reading context.
• **Prompts:** Predefined, reusable template prompts that assist in structuring specific interactions.

SECTION 2

Deployment & Transport Patterns

• **STDIO Transport:** Runs the MCP server as a local subprocess communicating over standard input/output streams (`stdin`/`stdout`). It delivers ultra-low latency and zero network overhead, making it ideal for local development and single-machine setups.
• **SSE (Server-Sent Events) Transport:** Operates the MCP server as an HTTP service streaming responses over long-lived connections (`/sse`). This supports multi-client access, remote cloud deployments, and shared microservices.
• **Hybrid Setups:** Agents can connect to multiple MCP servers simultaneously, blending local STDIO servers (e.g., local files or Git) with remote SSE servers (e.g., Slack or web search APIs) within a single session.

SECTION 3

Building and Testing MCP Servers

• **FastMCP & Schema Generation:** Wrapping a Python function with the `@mcp.tool()` decorator automatically turns its signature, docstrings, and return annotations into the tool schema that the LLM inspects.
• **The MCP Inspector:** Developers can debug servers using the `mcp dev` command line tool. The MCP Inspector launches an interactive web UI to inspect tool schemas, execute tool calls manually, and view raw JSON-RPC traffic.
• **Language-Agnostic Interoperability:** Because MCP communicates via standardized JSON-RPC 2.0 over the wire, a Python agent can consume a Node.js server (e.g., `@modelcontextprotocol/server-filesystem` via `npx`) without needing cross-language bridges or bindings.

SECTION 4

Assistants vs. Autonomous Agents

• **Assistants vs. Agents:** Assistants execute single user requests, often pausing for human approval after every step. Autonomous agents operate in a continuous loop—calling tools, observing outcomes, updating plans, and taking subsequent actions without interrupting the user.
• **Powering the Five Functional Layers:** While MCP is primarily used for tools and actions, it can augment all five agent layers:
• **Tools & Actions Layer:** GitHub, Slack, or File System servers.
• **Reasoning & Planning Layer:** Sequential Thinking servers that act as a step-by-step reasoning scratchpad.
• **Knowledge & Memory Layer:** Postgres, vector stores (e.g., Qdrant), or memory graph servers.
• **Evaluation & Feedback Layer:** LLM-as-judge or rubric scoring servers.

SECTION 5

Converting Internal Tools to MCP Servers

• **Separation of Concerns:** Moving internal python functions into a dedicated MCP server decouples the agent's logic from implementation details.
• **Reusability & Safety:** The agent only sees the tool's schema, inputs, and outputs. This allows the underlying backend (such as swapping an in-memory list for a relational database) to change without modifying any agent code.